

D3Rec Codebase – Simple Explanations

kerem

November 25, 2025

Contents

1	preprocessing.py — Simple Explanation	2
2	dataset_load.py — Simple Explanation	3
3	model_utils.py — Simple Explanation	4
4	models.py — Simple Explanation	6
5	utils_train.py — Simple Explanation	9
6	train.py — Simple Explanation	11
7	inference.py — Simple Explanation	13
8	Overall System Summary — Simple Explanation	15

1 preprocessing.py — Simple Explanation

This file is basically the "cleaning and preparing the data" part of the whole project. It does not train anything and it does not do any machine learning. Its only job is to take the raw dataset and turn it into something the model can use.

Here is what it does in a very simple way:

- It loads the raw data (user–item interactions, ratings, categories).
- If a prepared version of the data already exists, it loads that instead (this is the cache file).
- It removes bad or useless data:
 - drops very low ratings,
 - removes users who have almost no interactions,
 - removes items almost nobody interacted with.
- It turns all user IDs and item IDs into numbers starting from 0.
- It sorts every user's history by time (older first, newer last).
- It splits each user's history into:
 - 60% training,
 - 20% validation,
 - 20% test.
- It creates three user–item matrices:
 - train matrix,
 - validation matrix,
 - test matrix.

These are sparse matrices (mostly zeros).

- It looks at what categories each user interacted with and makes a "preference vector" for every user. This is basically:

"How much this user likes each category, as probabilities."
- It builds the item–category matrix F . This matrix says:

"Item i belongs to category c ."
- Finally, it saves everything into one single file so that next time, the program can load it instantly.

In short: this file takes messy raw data and turns it into clean numerical tensors that the diffusion model can understand. Nothing more, nothing less. It is the "data preparation" step of the whole system.

2 dataset_load.py — Simple Explanation

This file takes the cleaned data from `preprocessing.py` and turns it into a PyTorch-friendly format. The model cannot directly use the processed data, so this file creates small “dataset objects” that PyTorch can read in batches.

Here is the logic in simple terms:

1. The D3RecData class

This class represents a single user as one data point.

When the model asks:

“Give me the next user so I can train.”

This class answers by giving:

- the user’s item vector (a long array of 0/1 values showing which items they interacted with),
- the user’s category preference vector (probabilities for each category),
- the same preference vector again (used as a target for prediction).

The two important functions inside the class:

- `__len__` – tells PyTorch how many users there are.
- `__getitem__` – given a user index, returns that user’s data.

This is the basic structure all PyTorch training loops expect.

2. The load_data function

This is the main function of the file. Its job is to prepare everything needed for training and testing.

Here is what it does step by step:

- It calls `PreProcess(...)` to load the cleaned data.
- It creates three dataset objects:
 - a training dataset,
 - a validation dataset,
 - a test dataset.

Each one is a `D3RecData` object.

- It converts the item–category matrix F into a PyTorch tensor.

Finally, it returns five things:

- the full processed dataset object,
- the training dataset,
- the validation dataset,
- the test dataset,
- the item–category matrix F .

These five pieces are exactly what the training and inference code need.

3. Summary

In very simple words:

preprocessing.py cleans the raw data. **dataset_load.py** turns that cleaned data into PyTorch datasets that the model can train on.

Nothing complicated happens here: this file is just the “bridge” between the cleaned data and the actual training loop.

3 model_utils.py — Simple Explanation

This file contains small helper neural network pieces that the main model (D3Rec) uses inside **models.py**. You can think of this file as a toolbox: it does not do any training by itself, it just provides useful building blocks.

Below is what each part does in very simple terms.

1. The MyMLP class

This is a basic “multi-layer neural network”. It takes an input vector and transforms it through several linear layers. You can imagine it as:

input $\rightarrow$ linear $\rightarrow$ activation $\rightarrow$ linear $\rightarrow$ activation $\rightarrow$... $\rightarrow$ output.

The model uses this MLP to learn how to turn a noisy vector into a cleaner one. There is nothing fancy here: it is simply a stack of linear layers with activation functions.

2. The Encoder class

The encoder takes a user–item vector and produces a “hidden representation”. This hidden vector is smaller than the original one.

In simple words:

It compresses the long item vector into a smaller, dense vector that still keeps the important information.

Nothing magical happens: it’s just a linear layer and sometimes dropout.

3. The Decoder class

The decoder does the opposite of the encoder.

It takes the compressed hidden vector and tries to rebuild the original item vector.

In simple words:

hidden vector $\rightarrow$ try to predict the user's item preferences.

This is used during reconstruction when the model tries to produce x_0 .

4. CateEncoder (category encoder)

This part encodes the category information of the items.

It takes the item-category matrix F and processes it through a small network so the model can learn better category features.

In very simple terms:

“Teach the model what categories look like.”

This is used when predicting category preferences.

5. CateDecoder (category decoder)

This piece tries to reconstruct category scores from the model's hidden state.

The idea is:

If the model really understands the user's behaviour, it should also understand what categories they like.

So this decoder produces a predicted category vector.

6. cate_id_to_onehot helper

This is a tiny helper function.

Given a list of category IDs for an item, it produces a one-hot vector (a vector of zeros with 1s at the positions of the item's categories).

This is needed so the model can easily use category information.

7. Overall role of this file

In very short words:

model_utils.py gives D3Rec all the small neural network tools it needs.

It contains:

- MLP blocks,
- encoder/decoder parts,

- category-related methods,
- small convenience functions.

Without this file, the main model would be too messy and hard to read, because all the small components would be mixed in one place.

4 `models.py` — Simple Explanation

This file contains the main brains of the whole project. It defines two big things:

- the Diffusion class (how to add noise and remove noise),
- the D3Rec model (the neural network that predicts the clean signal).

Everything that happens during training and inference depends on these two. Below is the simple explanation of each part.

1. The Diffusion class

This class controls how we turn a clean user–item vector x_0 into a noisy version x_t , and how we later go back from x_t to x_0 .

Think of it like this:

“We gradually destroy the data by adding noise, then train a model to undo that damage.”

Inside the class, we have:

1.1 Beta and Alpha Schedules

The numbers called `beta` and `alpha` control how much noise we add. This file supports several kinds of noise schedules:

- linear,
- cosine,
- exponential,
- etc.

You can think of a noise schedule like:

“How strong should the noise be at step 1? How strong should it be at step 50? How strong at step 100?”

No need to know the math; it’s just a recipe for noise strength.

1.2 Adding Noise: `get_noised_interaction`

This function takes the clean vector x_0 and a timestep t , and returns a noisy version x_t .

In simple words:

$x_t = x_0$ mixed with random noise depending on t .

The bigger t is, the more noise you get.

1.3 Sampling Steps: `sample_steps`

During training, we do not always use the same t . This function picks random timesteps, but not all timesteps are equally likely. Harder timesteps (where the model struggles) get picked more often.

Idea:

“Spend more time training on difficult noise levels.”

2. The D3Rec Model

This is the main neural network that tries to remove noise. Given a noisy input x_t and the timestep t , it predicts:

- the cleaned vector x_0 ,
- the user’s category preference,
- the item-category matrix prediction,
- an orthogonality loss (to keep internal features independent).

So one forward pass returns four things.

2.1 Time Embedding

The model does not magically know what timestep t means. So we turn t into a vector using a small embedding network.

Imagine saying:

“Here is a vector telling you how noisy this input is.”

2.2 Dropout on Item Vector

Sometimes we randomly drop some parts of x_t (set them to zero). This helps the model learn to be more robust.

Simple idea:

“If you remove some information, the model should still be able to guess correctly.”

2.3 Encoder and Decoder

The model has:

- an encoder: turns the noisy vector into a hidden vector,
- a decoder: turns the hidden vector back into an item vector.

The decoder output is the predicted clean vector $\hat{x}_0$.

This is the main output the training loss uses.

2.4 Category Prediction

The model also tries to predict the user’s category preference. It has a separate head (a small neural network) for this.

Why? Because category information helps improve recommendations.

2.5 Category Matrix Prediction

The model also learns to predict the item–category matrix F . This forces the hidden representation to contain category-related structure.

2.6 Orthogonality Loss

The model splits its hidden vector into two halves. It wants these halves to be unrelated.

This avoids the model cheating by stuffing all information into one part.

We compute an extra “orthogonality loss” to force the two halves to be independent.

In simple words:

“Make part A and part B learn different things.”

3. The Forward Pass

When you call:

```
model(x_t, t, prob, prob_mask)
```

the following happens:

1. Add dropout to x_t .
2. Convert t into an embedding.
3. Encode x_t into a hidden vector.
4. Decode the hidden vector into $\hat{x}_0$.
5. Predict category preferences.
6. Predict category matrix.
7. Compute the orthogonality loss.

The model returns everything the training code needs.

4. Summary

In the simplest possible words:

Diffusion decides how to add and remove noise.

D3Rec learns how to remove that noise and recover user preferences.

This file defines the core of the whole system. Without these classes, nothing else can run.

5 `utils_train.py` — Simple Explanation

This file contains all the code that computes the losses and trains the model for one epoch. It is basically the “training math” of the whole project. It does not load data and it does not define the model — it only tells the model **how to learn**.

Here is the simple breakdown.

1. Category Loss

The function `compute_cate_loss` checks how well the model predicts the user’s category preferences.

Steps:

- Take the model’s predicted item vector.
- Multiply it with the item–category matrix.
- This gives a predicted category distribution.
- Compare it with the real category preference vector using MSE.

In simple words:

“Does the model guess which categories the user likes?”

2. Reweighting Loss

The function `reweight_loss` gives different weights to different items.

Reason:

“Some items should matter more than others when computing the loss.”

It uses the user’s category preferences to decide which items are more important.

3. Reconstruction Loss

The function `compute_recon_loss` is the main loss. It checks how close the predicted clean vector $\hat{x}_0$ is to the real clean vector x_0 .

It does:

- Compute $(\hat{x}_0 - x_0)^2$ for every item.
- Apply the item weights (from `reweight_loss`).
- Average over all items.
- Optionally apply SNR weights depending on the timestep.
- Update the importance-sampling history.

This is the most important part of training.

4. Total Loss

The function `calculate_loss` combines everything:

- Pick some timesteps.
- Add noise to x_0 to get x_t .
- Mask some category info.
- Run the model forward.
- Compute:
 - reconstruction loss,
 - category loss,
 - category-matrix loss (emb loss),
 - orthogonality loss.
- Add them all together.

This gives the final loss used for gradient descent.

5. Training One Epoch

The function `train_one_epoch` runs one full pass over the training data.

For each batch:

- Move data to GPU.
- Compute loss.
- Zero the gradients.
- Backpropagate.
- Update the model parameters.

At the end, it returns the average loss.

6. Summary

In simple words:

`utils_train.py` is the place where all training calculations happen. It tells the model how bad it is, how to improve, and updates everything.

It is the “learning engine” of the whole project.

6 `train.py` — Simple Explanation

This file is the main “training controller” of the whole project. It does not contain any math or model details. Instead, it connects all the other parts together and starts the actual training process.

Think of it like the guy who says:

“Load the data, create the model, train it, check the results, save the best model.”

Here is what happens step by step.

1. Reading the Training Settings

The function `get_args_parser()` defines all the options you can pass when starting training.

Things like:

- `--epochs` (how many epochs),
- `--batch_size`,
- `--lr` (learning rate),
- `--dataset_name`,

- `--dropout`,
- and many more.

This function just sets up all training parameters.

2. The `main()` Function

This is where everything actually happens.

2.1 Load Data

It calls `load_data()`, which:

- loads the cleaned dataset,
- creates the train/valid/test PyTorch datasets,
- loads the item–category matrix.

After that, it creates `DataLoaders` that produce batches of users.

2.2 Create Diffusion and Model

It constructs:

- a Diffusion object (for noise handling),
- a D3Rec model (the actual neural net).

Both are moved to the GPU.

2.3 Create the Optimizer

It uses AdamW for training:

The optimizer updates the model weights using the loss.

2.4 Loop Over Epochs

This is the main training loop:

1. Train for one epoch using `train_one_epoch()`.
2. Every few epochs (based on `eval_freq`), run evaluation.
3. If the validation score improves, save the model.
4. If the validation score stops improving for a long time, stop early (early stopping).

Every epoch prints:

- the training loss,
- how long the epoch took.

2.5 Validation and Test

When evaluation happens, the code:

- runs the model on the validation users,
- checks Recall@20,
- if it's the best so far:
 - saves the test results,
 - optionally saves the model to disk.

The test set is always checked at the best validation result.

3. The `__main__` Block

When you run:

```
python train.py ...
```

This part executes:

1. Read arguments.
2. Fix random seed.
3. Decide whether to use GPU or CPU.
4. Find dataset and model paths.
5. Call `main()` to start training.

4. Summary

In the simplest possible way:

train.py loads the data, creates the model, trains it, checks validation performance, saves the best model, and prints the final test results.

It connects all other files into one working training pipeline.

7 inference.py — Simple Explanation

This file is used **after the model is trained**. It does not train anything. Its only job is:

“Load the best model, run it on test users, produce recommendations, and print the evaluation results.”

It is basically the “use the model” part of the whole project.
Here is the simple breakdown.

1. Loading Data

The file first loads the cleaned dataset using `load_data`:

- the test dataset,
- the user–item matrices,
- the item–category matrix.

It also creates a `DataLoader` so the test users are processed in batches.

2. Creating Diffusion and the Model

Just like in training, the code rebuilds:

- a Diffusion object (for reverse denoising),
- a D3Rec model.

They must be created exactly the same way as in training.

3. Loading the Best Saved Model

The file loads the trained model weights from disk. This makes the model ready to run on new data.

In simple words:

“Use the learned parameters from training.”

4. Running Evaluation

The core part is running the function `evaluate(...)`.

This function:

- takes each test user,
- denoises x_t back to x_0 using the diffusion model,
- gets the model’s predicted clean vector,
- picks the top recommended items,
- compares them to what the user actually clicked in the test set,
- computes metrics like:
 - Precision@K,
 - Recall@K,
 - HitRate,

- nDCG,
- MRR,
- Coverage,
- Entropy.

It does this for different **temperature** values. Temperature controls how “sharp” or “diverse” the model’s predictions are.

Small temperature = more confident recommendations. Large temperature = more diverse recommendations.

5. Printing Results

For each temperature value, the code prints all metrics. This lets you compare model performance under different sampling styles.

Example printed results look like:

Precision@10, Recall@20, nDCG@10, Coverage, Entropy, etc.

6. Summary

In the simplest possible way:

inference.py loads the best trained model, denoises test users, produces top-item recommendations, and prints evaluation metrics.

It is the “testing and recommendation” stage of the system. No learning happens here — the model is only being used.

8 Overall System Summary — Simple Explanation

This section gives a full, simple overview of how the entire system works. No math, no complicated words. Just the clean idea behind the whole project.

1. What the System Tries to Do

The goal of the system is very straightforward:

“Look at a user’s past interactions, learn their preferences, and recommend new items they might like.”

But instead of using a normal recommender model, we use a **diffusion model**. This means we add noise to the data and train a network to remove the noise. By learning to remove noise, the model learns the user’s real behaviour patterns.

2. The Pipeline in Five Steps

Here is the whole system from start to finish:

1. **Preprocessing** Clean the raw data, remove useless interactions, encode IDs, split into train/valid/test, calculate user category preferences, and build the item–category matrix. (Done in `preprocessing.py`.)
2. **Dataset Loading** Turn the cleaned data into small PyTorch dataset objects that can be fed batch-by-batch into the model. (Done in `dataset_load.py`.)
3. **Model + Diffusion** Build the diffusion noise schedule and the D3Rec neural network. D3Rec takes a noisy user vector x_t and tries to predict the clean x_0 . (Done in `models.py` and `model_utils.py`.)
4. **Training** For each batch:
 - choose a timestep,
 - add noise to the clean vector,
 - let the model try to remove the noise,
 - compute losses (reconstruction, categories, embeddings, orthogonality),
 - update the model weights.

After every few epochs, check validation performance and save the best model. (Done in `utils_train.py` and `train.py`.)

5. **Inference** Load the best model, run it on test users, clean the noisy inputs, get final item scores, pick the top items, and compute metrics like Recall@K. (Done in `inference.py`.)

3. How the Diffusion Idea Fits Here

The diffusion part works like this:

- Start with a clean user vector x_0 (what items they interacted with).
- Add controlled noise to get x_t .
- Train the model to remove the noise and predict x_0 back.
- If the model becomes good at this, it also becomes good at understanding what the user likes and what they might click next.

Removing noise becomes the way of learning preferences.

4. Why the Model Predicts More Than Just x_0

Besides the clean vector x_0 , the model also predicts:

- the user’s category distribution,
- the item–category matrix,
- an orthogonality signal to keep features separated.

These extra tasks help stabilize training and make the model better at capturing structure in the data.

5. What Happens at Test Time

During inference:

- we start with a noisy vector,
- the model removes noise step-by-step,
- the clean output gives item scores,
- we sort the scores and pick the top recommended items.

We then compare those items with what the user actually interacted with and compute metrics.

6. The Entire System in One Sentence

The project takes cleaned interaction data, adds noise to it, trains a model to remove that noise, and uses the cleaned predictions to recommend items and evaluate how well the model understands user behaviour.

This is the whole story in the simplest possible way.